DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No.: 9

Student Name: Probal Dhali
UID: 24BAI71013
Branch: AIT-CSE
Section/Group: B
Semester: 5TH
Date of Performance:
Subject Name: Competitive Coding – II
Subject Code: 24CSP-305

Experiment No: 9.1

Aim: To write a C++ program to find the minimum number of coins required to make a given amount.

Objective: To understand dynamic programming and solve the coin change problem efficiently.

Problem Statement: Given an array of coin denominations and an integer amount, find the minimum number of coins needed to make that amount. If the amount cannot be made using the given coins, return -1. You may use each coin denomination any number of times.

LeetCode Link: https://leetcode.com/problems/coin-change/submissions/2182149368/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Initialize a DP array of size amount + 1 with amount + 1.
2. Set dp[0] = 0.
3. Iterate from 1 to amount.
4. For each amount, check every coin denomination.
5. If the coin value is less than or equal to the current amount, update the minimum coin count.
6. Return dp[amount] if a solution exists; otherwise, return -1.

Code:

```
#include <bits/stdc++.h>

using namespace std;

class Solution {

public:

   int coinChange(vector<int>& coins, int amount) {

      vector<int> dp(amount + 1, amount + 1);

      dp[0] = 0;

      for (int i = 1; i <= amount; i++) {

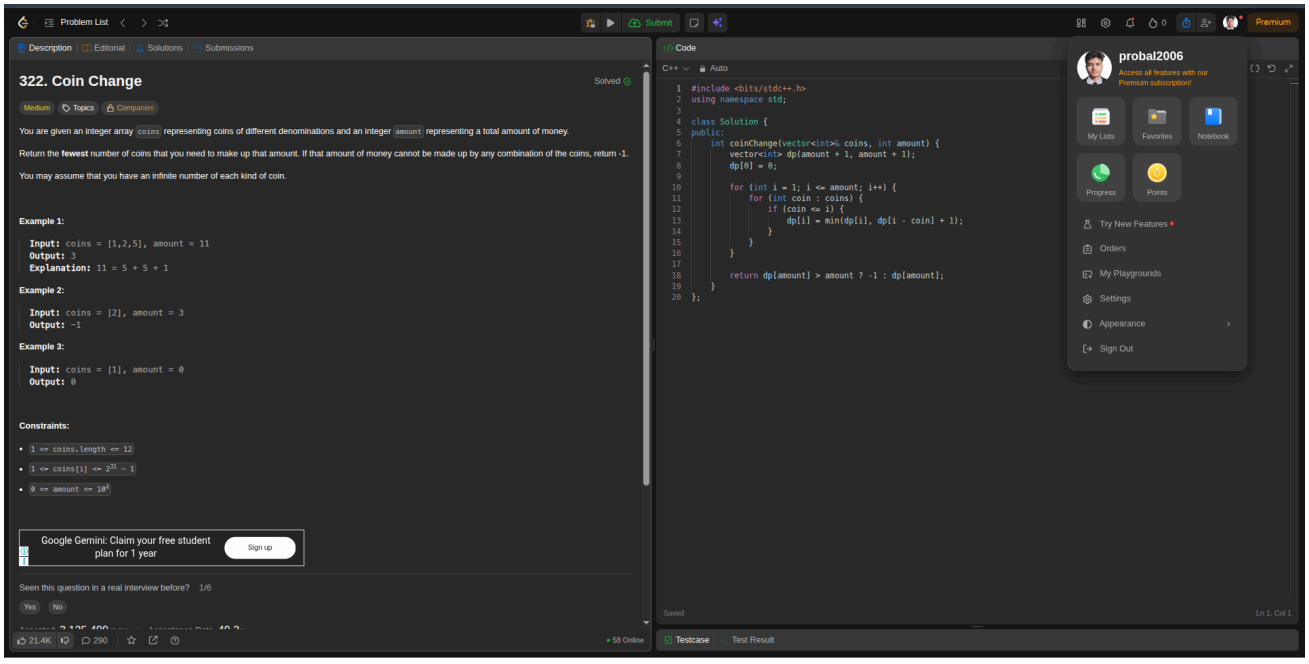
         for (int coin : coins) {

            if (coin <= i) {

               dp[i] = min(dp[i], dp[i - coin] + 1);
```

```
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
  }
};
```

Output:



Observations/Outcome: The program successfully finds the minimum number of coins required to make the given amount using dynamic programming.

Time Complexity: $O(A \times N)$

Space Complexity: $O(A)$

Learning outcomes :

- Understand the concept of dynamic programming.
- Solve optimization problems using a DP array.
- Find the minimum number of coins for a given amount.
- Analyze time and space complexity.

Experiment No.: 9.2

Aim: To write a C++ program to find the length of the Longest Common Subsequence (LCS) of two strings.

Objective: To understand dynamic programming and identify the longest subsequence common to two given strings.

Problem Statement: Given two strings text1 and text2, find the length of their longest common subsequence. A subsequence is formed by deleting zero or more characters without changing the order of the remaining characters.

LeetCode Link: https://leetcode.com/problems/longest-common-subsequence/description/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Initialize a DP table of size (m + 1) × (n + 1) with zeros.
2. Traverse both strings using nested loops.
3. If the characters match, set dp[i][j] = 1 + dp[i-1][j-1].
4. Otherwise, set dp[i][j] to the maximum of dp[i-1][j] and dp[i][j-1].
5. Return dp[m][n] as the length of the LCS.

Code:

```
#include <bits/stdc++.h>

using namespace std;

class Solution {

public:

   int longestCommonSubsequence(string text1, string text2) {

      int m = text1.size();

      int n = text2.size();

      vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

      for (int i = 1; i <= m; i++) {

         for (int j = 1; j <= n; j++) {

            if (text1[i - 1] == text2[j - 1]) {

               dp[i][j] = 1 + dp[i - 1][j - 1];

            } else {

               dp[i][j] = max(dp[i - 1][j],

                              dp[i][j - 1]);

            }

         }
```

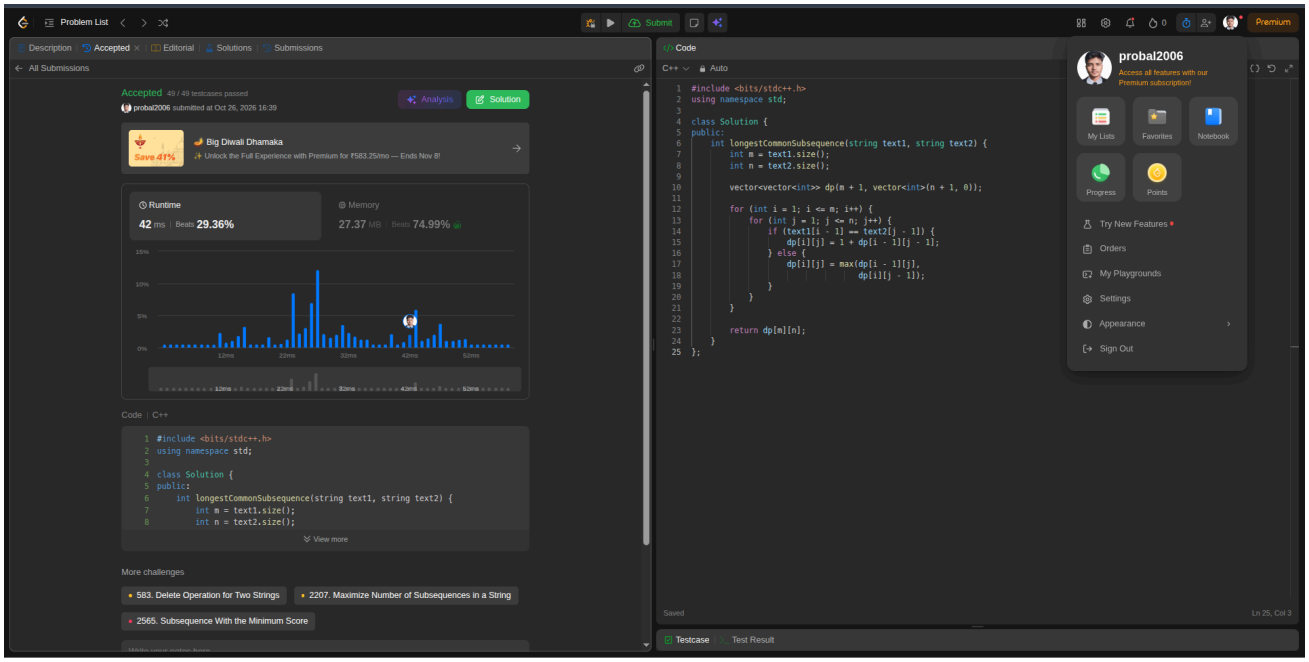
```
        }

        return dp[m][n];

    }

};
```

Output:



Observations/Outcome: The program successfully finds the length of the longest common subsequence using dynamic programming.

Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

Learning outcomes :

- Understand the concept of subsequences.
- Apply dynamic programming to string problems.
- Implement the LCS algorithm in C++.
- Analyze time and space complexity.